

From a weighted sum to a neuron

A neural network becomes expressive when affine maps are separated by nonlinear activations.

THE QUESTION

What does a neural network add to a linear model, and why can it solve problems that no single line can?

THE INTUITION

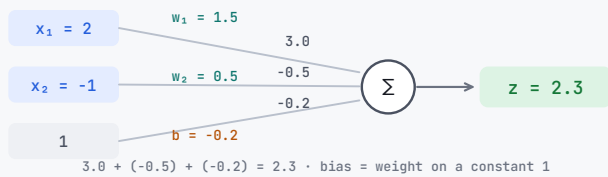
A weighted sum can tilt and shift a line but never bend it, and stacking weighted sums gives one weighted sum. An activation between layers adds the bend, so hidden units become new features.

KEY TAKEAWAY

Affine map = choose a direction and offset. Activation = bend. Composition = representation. Without the nonlinearity, depth adds nothing.

input x → weighted sum + bias → activation ϕ → hidden features h → output scores → task map

1 The linear starting point



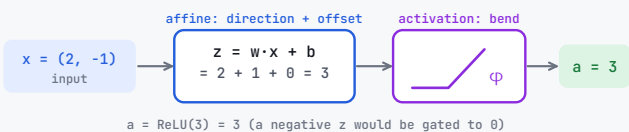
One scalar output starts as $z = w^T x + b$ with $x, w \in \mathbb{R}^d$ and $b, z \in \mathbb{R}$. Each edge contributes weight $\times$ incoming value; the node adds the contributions and the bias. The bias is a weight on an extra constant feature equal to 1.

2 Batch form and shape audit



Stack n examples as rows of $X \in \mathbb{R}^{n \times d}$: $\hat{y} = Xw + b1$. Or append a ones column and stack the bias into θ : $\hat{y} = \tilde{X}\theta$, with shapes $(n \times (d+1))((d+1) \times 1) \rightarrow (n \times 1)$.

3 A neuron adds a nonlinear stage



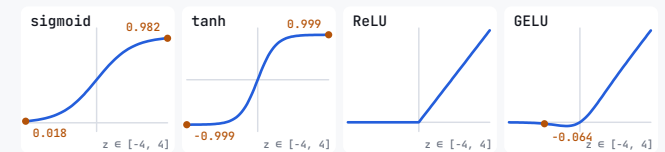
$z = w^T x + b$, $a = \phi(z)$. The affine stage chooses an input direction and offset; the activation gates or reshapes the resulting score.

4 Output-layer contract

task	raw output	final map
regression	one or more scores	identity / task constraint
binary	one logit z	sigmoid $\sigma(z)$
K classes	K logits	softmax over K

The hidden representation can stay the same; the number and meaning of output nodes follows the target.

5 Activation map



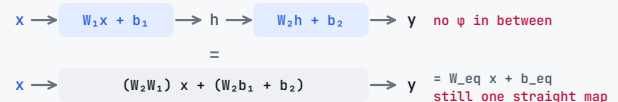
name	rule / range	use and warning
sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	probability-like switch; saturates
tanh	$\tanh(z) \in (-1, 1)$	signed switch; saturates
ReLU	$\max(0, z)$	simple ramp; zero slope for $z < 0$
GELU	$\approx z\sigma(1.702z)$	smooth, input-dependent gate

6 Saturation is a learning issue



A flat activation has a tiny derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, so $\sigma'(0) = 0.25$ but $\sigma'(6) \approx 0.0025$. The same input nudge moves the output about 100 $\times$ less in the saturated regime.

7 Why nonlinearity is essential



With $h = W_1 x + b_1$ and $y = W_2 h + b_2$:

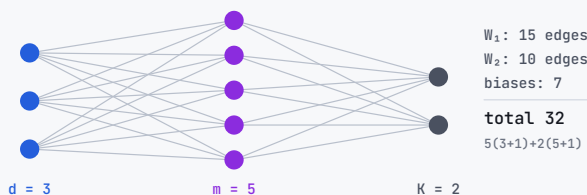
$$y = \underbrace{W_2 W_1}_{W_{eq}} x + \underbrace{W_2 b_1 + b_2}_{b_{eq}}$$

Without ϕ , extra layers add parameters but no new bends or learned feature shapes.

Read an MLP by its shapes and features

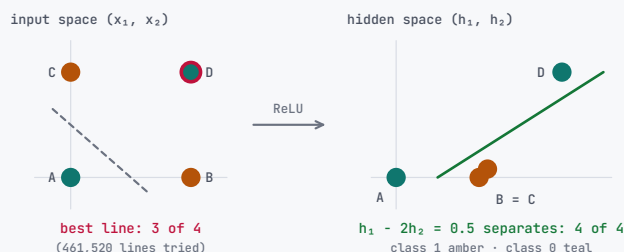
Hidden units learn coordinates in which the final prediction can be simpler.

8 One hidden layer



$h = \varphi(W_1 x + b_1)$, $z = W_2 h + b_2$, with input width d , hidden width m , K outputs: $W_1 \in \mathbb{R}^{m \times d}$, $b_1 \in \mathbb{R}^m$, $W_2 \in \mathbb{R}^{K \times m}$, $b_2 \in \mathbb{R}^K$. Shapes: $(m \times d)(d \times 1) \rightarrow m$, then $(K \times m)m \rightarrow K$. Trainable scalars = $m(d+1) + K(m+1)$.

9 XOR: features solve what one line cannot



One binary linear score has one straight boundary. Build two features, then one output:

$$h_1 = \text{ReLU}(x_1 + x_2), \quad h_2 = \text{ReLU}(x_1 + x_2 - 1),$$

$$z = 2h_1 - 4h_2 - 1, \quad \hat{y} = \mathbb{1}[z \geq 0].$$

Result: $(0, 0) \rightarrow 0$, $(1, 0) \rightarrow 1$, $(0, 1) \rightarrow 1$, $(1, 1) \rightarrow 0$ (9 parameters). In (h_1, h_2) space, B and C land on the same point, so one line now works.

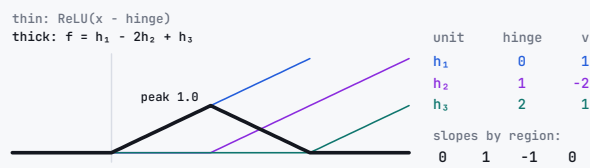
Exact on the four Boolean points does not mean the same rule labels every point of a filled XOR region: these two units agree with the filled target on 75% of the unit square. Four ReLUs match it everywhere off the two midlines. State the input domain.

10 Width and depth answer different needs



Width: more units in one layer, more features at the same stage. **Depth:** more successive transformations; later features reuse earlier ones. Above, two one-hinge units composed give a function with two bends. A layer receives the previous representation, not the original input, unless a skip connection supplies it.

11 ReLU networks assemble hinges

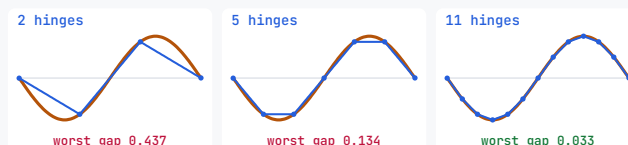


One shifted ReLU is a hinge, $h_j(x) = \text{ReLU}(w_j x + b_j)$; a linear output adds them:

$$g(x) = c + \sum_j v_j \text{ReLU}(w_j x + b_j).$$

Input weights and biases place the hinges; output weights v_j set the slope change at each. In higher dimensions ReLU units fold space along hyperplanes.

12 Universal approximation, read carefully



For continuous f on a closed, bounded region D and any $\varepsilon > 0$, a wide enough finite network with a suitable nonlinear activation can reach $\sup_{x \in D} |f(x) - g(x)| < \varepsilon$. Above, $\sin$ (amber) and hinge fits (blue): 11 hinges already beat $\varepsilon = 0.05$ on $[-\pi, \pi]$. This is **existence and capacity**. It does not promise that gradient training finds the network, that it is small, or that it generalizes beyond D .

The fits are constructed interpolants with fixed knots, not trained networks.

13 Fast architecture audit

- ✓ Write the shape of every value and parameter.
- ✓ Point to the nonlinearity between affine layers.
- ✓ Explain each hidden layer as a learned feature map.
- ✓ Match the output width and final map to the task.
- ✓ Separate representability from trainability.

14 Pause and think

Why can't more linear layers solve XOR? They compose to one affine map, so the boundary is still one line: the best gets 3 of 4.

Where does hinge j sit? At $x = -b_j/w_j$; the bias moves it, v_j sets its slope change.

5 hinges miss $\sin$ by 0.134. Fix? More units (11 give 0.033), but only on the bounded D .

Affine map = select and shift a direction • activation = bend • composition = representation

Numbers from the Lecture 4 slides and notebooks (XOR hand weights, hinge examples, $\sin$ fits). Slides and summary videos: nipunbatra.github.io/dl-2026.